1.3 R Essentials

Objects in R

An object is a named container that stores something, such as:

- A number
- A vector of values
- A table of data
- The result of a calculation

Objects allow you to reuse results without retyping them.

Creating objects with the assignment operator <-

The most common way to create an object in R is using <-, which you can read as “store this in”.

x <- 10

Explanation:

- x is the name of the object.
- <- assigns the value on the right to the name on the left.
- 10 is a numeric value.
- After running this line, x exists and can be reused.

To view the value stored in the object x, type x in the console.

R prints the value stored in x. Even though x contains only one value, it is a vector. Scalars are just one-element vectors in R. Vectors can contain multiple values.

Important point:

Variable names are case sensitive in R. x is not the same as X.

Arithmetic in R

R can be used like a calculator. You type an expression, R evaluates it, and the result is returned.

Operator	Meaning	Example
+	Addition	10 + 5
-	Subtraction	10 - 5
*	Multiplication	10 * 5
/	Division	10 / 5
^	Power	10 ^ 2

Order of operations (BEDMAS)

R follows standard mathematical rules for the order of operations.

2 + 3 * 4

Explanation:

- Multiplication is performed before addition.
- 3 * 4 is evaluated first, giving 12.
- Then 2 + 12 is calculated.
- The result is 14.

If you want to control the order, use parentheses.

(2 + 3) * 4

Explanation:

- The expression inside the parentheses is evaluated first.
- 2 + 3 becomes 5.
- 5 * 4 gives 20.

This is important in finance, where formulas often depend on correct grouping of terms.

Functions: How R Performs Tasks

A function is similar to an Excel formula:

- Excel: =AVERAGE(A1:A10)
- R: mean(values)

A function always has:

- A name
- Parentheses containing inputs

Creating collections with c()

The first function you will encounter is c().

prices <- c(100, 101.5, 99.8, 103.2)

prices

Explanation:

- c() stands for “combine”.
- It combines multiple values into a single object.
- The result is stored in an object called prices.

This object is a **vector**, which is the simplest and most common way to store financial data.

Mathematical Functions

R includes many built-in mathematical functions. These are frequently used in finance for transformations and growth calculations.

Function	Purpose	Example
log(x)	Natural logarithm	log(1.05)
exp(x)	Exponential function	exp(2)
sqrt(x)	Square root	sqrt(49)
abs(x)	Absolute value	abs(-12)
sin(x)	Sine (radians)	sin(0)

These functions take a number (or a vector of numbers) as input and return a transformed value.

Statistical Functions

Statistical functions summarise data, which is central to financial analytics.

First, create a small vector of returns.

returns <- c(0.02, -0.01, 0.03, 0.015)

Now apply summary functions.

Function	What it returns
mean(returns)	Average return
median(returns)	Middle value
min(returns)	Smallest value
max(returns)	Largest value
nrow(df)	Number of rows in a data frame

These functions help you describe performance, variability, and extremes in financial data.

Combining Operators and Functions

In practice, operators and functions are almost always used together.

prices <- c(100, 101.5, 99.8)

scaled_returns <- prices / 100 - 1

scaled_returns

Explanation:

- prices / 100 divides each price by 100.
- Subtracting 1 converts the ratios into growth rates.
- The calculation is applied element by element.

You can then summarise the result:

mean_scaled_return <- mean(scaled_returns)

mean_scaled_return

Explanation:

- mean() computes the average of the vector.
- The result is stored for later use.

Storing Results and the Environment Pane

Whenever you use <-, you store a result as an object.

avg_return <- mean(returns)

That object appears in the **Environment pane** in RStudio. This allows you to:

- See what data and results you have created
- Reuse values without recalculating
- Keep your analysis organised and reproducible

Data Structures: Vectors and Data Frames

R stores data in different structures. In this course, we focus on two.

Vectors

A vector stores a single variable across multiple observations.

quantity <- c(200, 220, 210, 230)

quantity

Explanation:

- All values are of the same type.
- Vectors commonly represent prices, quantities, or returns.

Data frames

A data frame stores multiple related variables together in a table.

```
df <- data.frame(
  date = c("2026-01-29", "2026-01-30", "2026-02-02", "2026-02-03"),
  price = c(100.0, 101.5, 99.8, 103.2),
  quantity = c(200, 220, 210, 230)
)
df
```

Explanation:

- Each column is a vector.
- Each row is one observation.
- This structure mirrors how financial data is organised in practice.

Packages: Extending R

Base R provides core functionality. **Packages** extend R with additional tools written by others.

Packages are essential in financial analytics because they allow you to:

- Work with time series data
- Download market and economic data
- Visualise trends
- Apply established analytical methods

Installing and loading a package

```
install.packages("tidyverse")
library(tidyverse)
```

Explanation:

- install.packages() downloads the package (usually done once).
- library() makes the package available in your current session.

Creating New Variables with mutate()

The mutate() function is used to create new columns in a data frame.

```
df <- df %>%
  mutate(revenue = price * quantity)
```

df

Explanation:

- The **pipe operator** (%>%) is used to pass the result of one step directly into the next step, making code easier to read from top to bottom.
- In RStudio, the keyboard shortcut for the pipe operator %>% is:

Ctrl + Shift + M (Windows)
Cmd + Shift + M (macOS

- %>% passes the data frame into mutate().
- revenue = price * quantity defines a new variable using existing columns.
- The result is added as a new column.

This reflects a common financial task: creating new measures from existing data.

Subsetting and Logical Comparisons

Financial analysis often requires selecting specific observations.

df[df$price > 100,]

Explanation:

- df$price: The $ operator extracts a single column from a data frame.
- df$price returns the entire price column as a vector.

df$price > 100

Explanation:

- df$price > 100 creates a logical vector (TRUE/FALSE for each row). Where the price is greater than 100, R returns TRUE; where the price is not greater than 100, R returns FALSE.
- Example output might look like:
  FALSE TRUE TRUE FALSE ...

df[df$price > 100 ,]

Explanation:

- R keeps only the rows where the condition is TRUE.
- The empty space after the comma means “keep all columns”.
- You can also compare values directly.

df$revenue > 20000

Explanation:

- Returns TRUE or FALSE for each row.

- Logical comparisons are commonly used for filtering and decision rules.

Reading

This is an exceptional resource for learning the Tidyverse, which is a collection of open-source R packages designed specifically for data science.

R for Data Science (2e) (https://r4ds.hadley.nz/)

Here is a video to get you started with R

https://www.youtube.com/watch?v=yZ0bV2Afkjc (https://www.youtube.com/watch?v=yZ0bV2Afkjc)

(https://www.youtube.com/watch?v=yZ0bV2Afkjc)